CRUX: HOW TO ADD LOCKS TO DATA STRUCTURES

When given a particular data structure, how should we add locks to it, in order to make it work correctly? Further, how do we add locks such that the data structure yields high performance, enabling many threads to access the structure at once, i.e., **concurrently**?

Concurrent Counters

- Counter is one of the simplest data structure.
- A non-concurrent counter implementation:

```
1  typedef struct __counter_t {
2      int value;
3  } counter_t;
4
5  void init(counter_t *c) {
6      c->value = 0;
7  }
8
9  void increment(counter_t *c) {
10     c->value++;
11 }
12
13 void decrement(counter_t *c) {
14     c->value--;
15 }
16
17 int get(counter_t *c) {
18     return c->value;
19 }
```

Figure 29.1: **A Counter Without Locks**

Simple But Not Scalable

How make this thread safe?

```
1  typedef struct __counter_t {
2      int             value;
3      pthread_mutex_t lock;
4  } counter_t;
5
6  void init(counter_t *c) {
7      c->value = 0;
8      Pthread_mutex_init(&c->lock, NULL);
9  }
10
11 void increment(counter_t *c) {
12     Pthread_mutex_lock(&c->lock);
13     c->value++;
```

```
14      Pthread_mutex_unlock(&c->lock);
15  }
16
17  void decrement(counter_t *c) {
18      Pthread_mutex_lock(&c->lock);
19      c->value--;
20      Pthread_mutex_unlock(&c->lock);
21  }
22
23  int get(counter_t *c) {
24      Pthread_mutex_lock(&c->lock);
25      int rc = c->value;
26      Pthread_mutex_unlock(&c->lock);
27      return rc;
28  }
```

Figure 29.2: A Counter With Locks

> The above code is how we do it. It simply adds a single lock, which is acquired when calling a routine that manipulates the data structure, and is released automatically as you call & return.

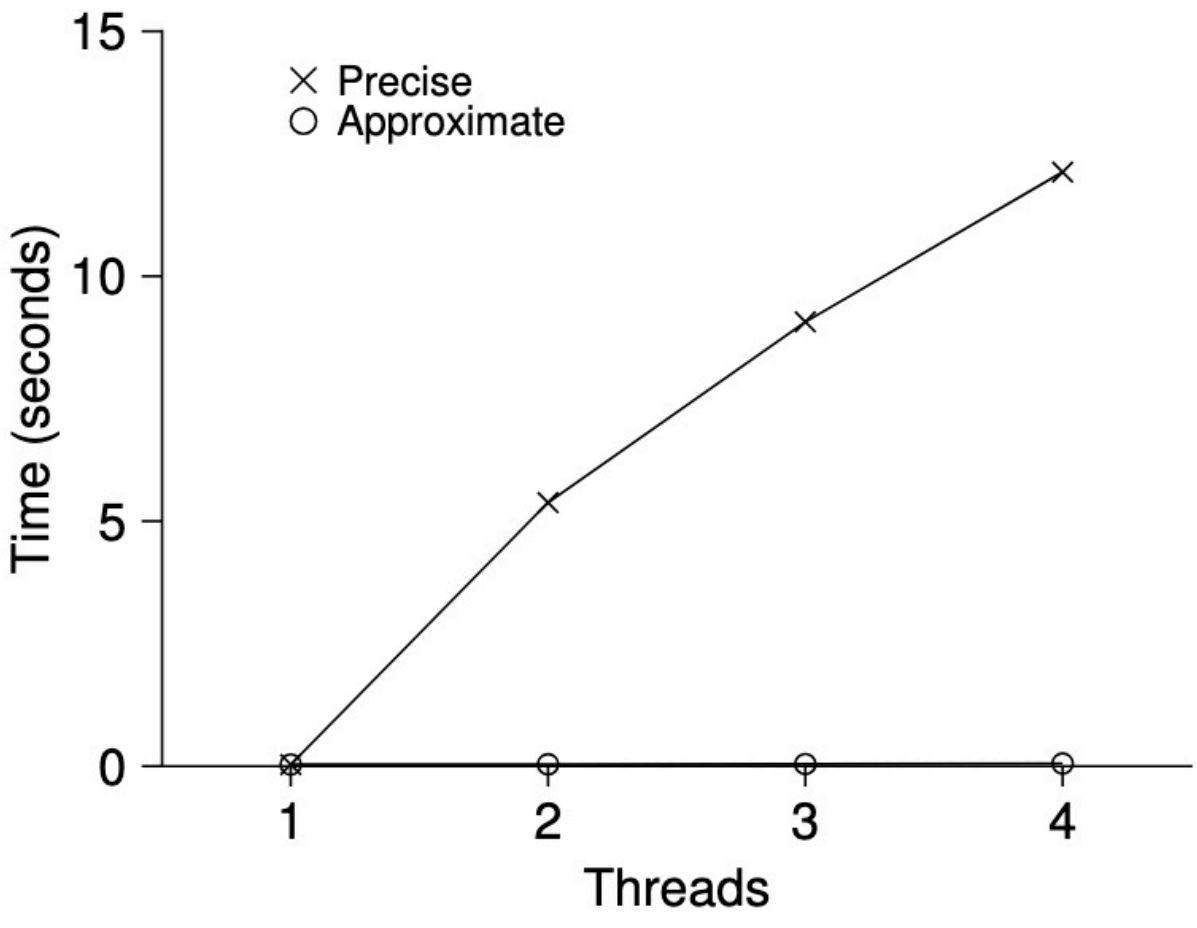


Figure 29.5: Performance of Traditional vs. Approximate Counters

Now, we have a concurrent data structure but it slow. In above figure,

> A single thread updates the counter 1M times in 0.03 sec but 2 threads each update counter 1M times take 5 secs.

Ideally, we want threads to complete just as quickly on multiple processors as the single thread does on one.

Scalable Counting

↳ Many techniques have been developed for this, one is approximate counter.

↳ It works by representing a single logical counter via numerous local physical counters, one per CPU core, as well as a single global counter.

For example, a machine with four CPUs, have four local counters & one global counter.

Additionally, there is one lock per local counter as there may be more than one thread on each core.

↳ Basic idea:

- When a thread running on a core wishes to increment the counter, it increments its local counter by acquiring corresponding local lock
- As each CPU has its own local counter, there is no contention, which makes it scalable.
- To keep global counter up to date, by acquiring the global lock, and incrementing it by local counter's value. The local counter is then reset to zero.
- How often, this local-to-global transfer takes place depends on S.

Large S: more scalable but further the global counter might be from actual count.

Small S: less scalable

The lower line in figure 29.5, shows performance with S = 1024

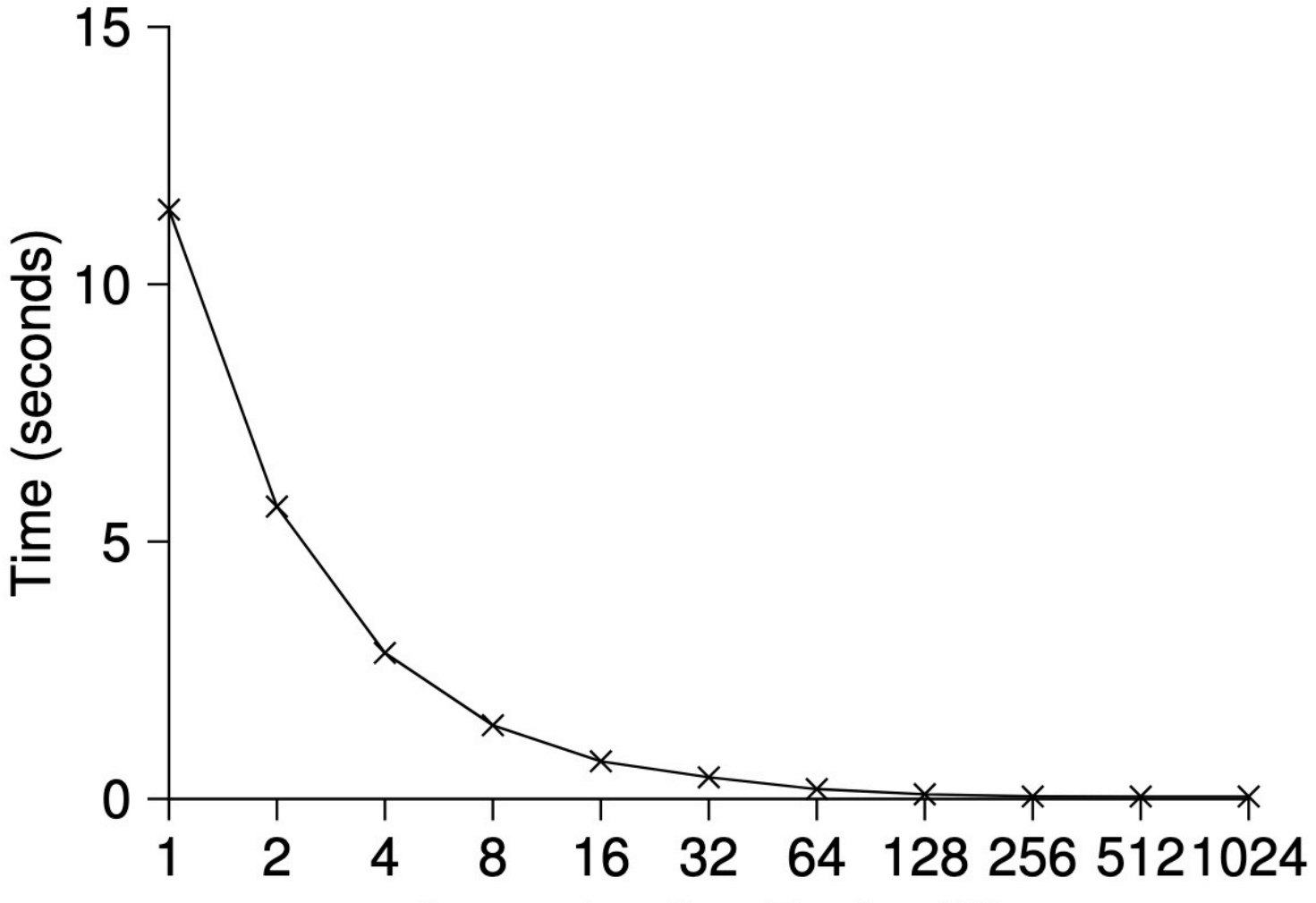


Figure 29.6: **Scaling Approximate Counters**

The above figure shows importance of S

```
1  typedef struct __counter_t {
2      int             global;          // global count
3      pthread_mutex_t glock;           // global lock
4      int             local[NUMCPUS]; // per-CPU count
5      pthread_mutex_t llock[NUMCPUS]; // ... and locks
6      int             threshold;       // update freq
7  } counter_t;
8
9  // init: record threshold, init locks, init values
10 //       of all local counts and global count
11 void init(counter_t *c, int threshold) {
12     c->threshold = threshold;
13     c->global = 0;
14     pthread_mutex_init(&c->glock, NULL);
15     int i;
16     for (i = 0; i < NUMCPUS; i++) {
17         c->local[i] = 0;
18         pthread_mutex_init(&c->llock[i], NULL);
19     }
20 }
21
22 // update: usually, just grab local lock and update
23 // local amount; once it has risen 'threshold',
24 // grab global lock and transfer local values to it
25 void update(counter_t *c, int threadID, int amt) {
26     int cpu = threadID % NUMCPUS;
27     pthread_mutex_lock(&c->llock[cpu]);
28     c->local[cpu] += amt;
29     if (c->local[cpu] >= c->threshold) {
30         // transfer to global (assumes amt>0)
31         pthread_mutex_lock(&c->glock);
```

```
32            c->global += c->local[cpu];
33            pthread_mutex_unlock(&c->glock);
34            c->local[cpu] = 0;
35        }
36        pthread_mutex_unlock(&c->llock[cpu]);
37    }
38
39    // get: just return global amount (approximate)
40    int get(counter_t *c) {
41        pthread_mutex_lock(&c->glock);
42        int val = c->global;
43        pthread_mutex_unlock(&c->glock);
44        return val; // only approximate!
45    }
```

Figure 29.4: **Approximate Counter Implementation**

Concurrent Linked Lists

```
1   // basic node structure
2   typedef struct __node_t {
3       int                 key;
4       struct __node_t         *next;
5   } node_t;
6
7   // basic list structure (one used per list)
8   typedef struct __list_t {
9       node_t                *head;
10      pthread_mutex_t     lock;
11  } list_t;
12
13  void List_Init(list_t *L) {
14      L->head = NULL;
15      pthread_mutex_init(&L->lock, NULL);
16  }
17
18  int List_Insert(list_t *L, int key) {
19      pthread_mutex_lock(&L->lock);
20      node_t *new = malloc(sizeof(node_t));
21      if (new == NULL) {
22          perror("malloc");
23          pthread_mutex_unlock(&L->lock);
24          return -1; // fail
25      }
26      new->key  = key;
27      new->next = L->head;
28      L->head   = new;
29      pthread_mutex_unlock(&L->lock);
30      return 0; // success
31  }
32
33  int List_Lookup(list_t *L, int key) {
34      pthread_mutex_lock(&L->lock);
35      node_t *curr = L->head;
36      while (curr) {
37          if (curr->key == key) {
38              pthread_mutex_unlock(&L->lock);
39              return 0; // success
40          }
41          curr = curr->next;
42      }
43      pthread_mutex_unlock(&L->lock);
44      return -1; // failure
45  }
```

Figure 29.7: **Concurrent Linked List**

if malloc fails, we need to free the lock.

can we rewrite the insert and lookup routines to remain correct under concurrent insert but avoid the case where the failure path also requires us to add the call to unlock?

Yes, we can rearrange the code so that lock surrounds the actual critical section assuming malloc is thread safe.

```
1  void List_Init(list_t *L) {
2      L->head = NULL;
3      pthread_mutex_init(&L->lock, NULL);
4  }
5
6  int List_Insert(list_t *L, int key) {
7      // synchronization not needed
8      node_t *new = malloc(sizeof(node_t));
9      if (new == NULL) {
10         perror("malloc");
11         return -1;
12     }
13     new->key = key;
14     // just lock critical section
15     pthread_mutex_lock(&L->lock);
16     new->next = L->head;
17     L->head   = new;
18     pthread_mutex_unlock(&L->lock);
19     return 0; // success
20 }
21
22 int List_Lookup(list_t *L, int key) {
23     int rv = -1;
24     pthread_mutex_lock(&L->lock);
25     node_t *curr = L->head;
26     while (curr) {
27         if (curr->key == key) {
28             rv = 0;
29             break;
30         }
31         curr = curr->next;
32     }
33     pthread_mutex_unlock(&L->lock);
34     return rv; // now both success and failure
35 }
```

Figure 29.8: **Concurrent Linked List: Rewritten**

Scalable Linked List:

Q We add a lock per node, this approach is called hand-over-hand locking.

When traversing the list, the code grabs the next node's lock & then releases the current node's lock.

Concurrent Queues

- There is the standard way of one big lock.
- Other is to have a two locks: one for head & one for tail.

The enqueue routine will only access tail lock & dequeue only head lock.

- Micheal & Scott added a dummy node that enables seperation of head & tail operations

```
1  typedef struct __node_t {
2      int                 value;
3      struct __node_t     *next;
4  } node_t;
5
6  typedef struct __queue_t {
7      node_t              *head;
8      node_t              *tail;
9      pthread_mutex_t     head_lock, tail_lock;
10 } queue_t;
11
12 void Queue_Init(queue_t *q) {
13     node_t *tmp = malloc(sizeof(node_t));
14     tmp->next = NULL;
15     q->head = q->tail = tmp;
16     pthread_mutex_init(&q->head_lock, NULL);
17     pthread_mutex_init(&q->tail_lock, NULL);
18 }
19
20 void Queue_Enqueue(queue_t *q, int value) {
21     node_t *tmp = malloc(sizeof(node_t));
22     assert(tmp != NULL);
23     tmp->value = value;
24     tmp->next  = NULL;
25
26     pthread_mutex_lock(&q->tail_lock);
27     q->tail->next = tmp;
28     q->tail = tmp;
29     pthread_mutex_unlock(&q->tail_lock);
30 }
31
32 int Queue_Dequeue(queue_t *q, int *value) {
33     pthread_mutex_lock(&q->head_lock);
34     node_t *tmp = q->head;
35     node_t *new_head = tmp->next;
36     if (new_head == NULL) {
37         pthread_mutex_unlock(&q->head_lock);
```

```
38            return -1; // queue was empty
39        }
40        *value = new_head->value;
41        q->head = new_head;
42        pthread_mutex_unlock(&q->head_lock);
43        free(tmp);
44        return 0;
45    }
```

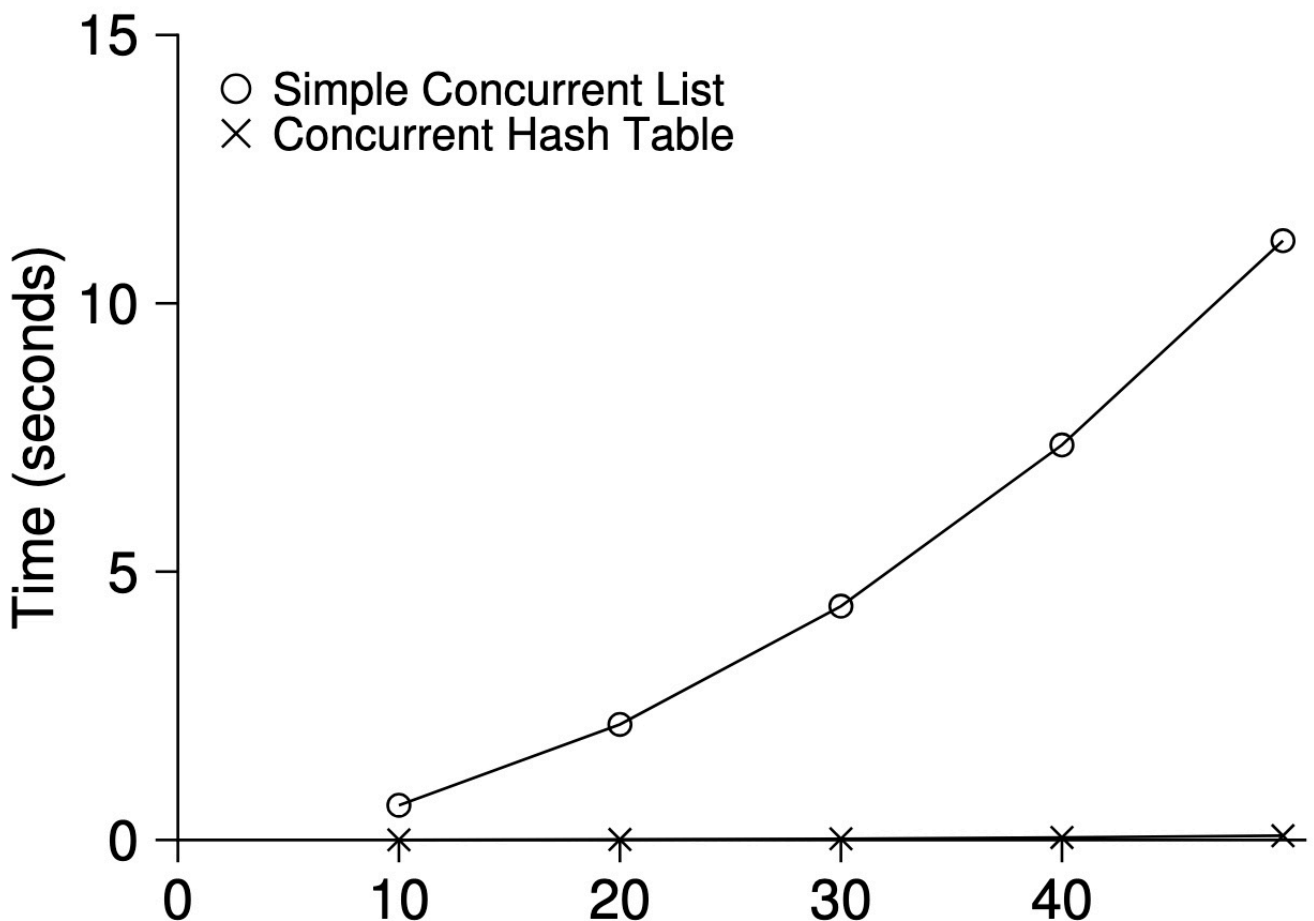
Figure 29.9: **Michael and Scott Concurrent Queue**

Concurrent Hash Table

The implementation below has fixed no. of buckets, & there is a lock per bucket

```
1   #define BUCKETS (101)
2
3   typedef struct __hash_t {
4       list_t lists[BUCKETS];
5   } hash_t;
6
7   void Hash_Init(hash_t *H) {
8       int i;
9       for (i = 0; i < BUCKETS; i++)
10          List_Init(&H->lists[i]);
11  }
12
13  int Hash_Insert(hash_t *H, int key) {
14      return List_Insert(&H->lists[key % BUCKETS], key);
15  }
16
17  int Hash_Lookup(hash_t *H, int key) {
18      return List_Lookup(&H->lists[key % BUCKETS], key);
19  }
```

Figure 29.10: **A Concurrent Hash Table**



Inserts (Thousands)

Figure 29.11: **Scaling Hash Tables**

Figure 29.11 (page 13) shows the performance of the hash table under concurrent updates (from 10,000 to 50,000 concurrent updates from each of four threads, on the same iMac with four CPUs). Also shown, for the sake of comparison, is the performance of a linked list (with a single lock). As you can see from the graph, this simple concurrent hash table scales magnificently; the linked list, in contrast, does not.